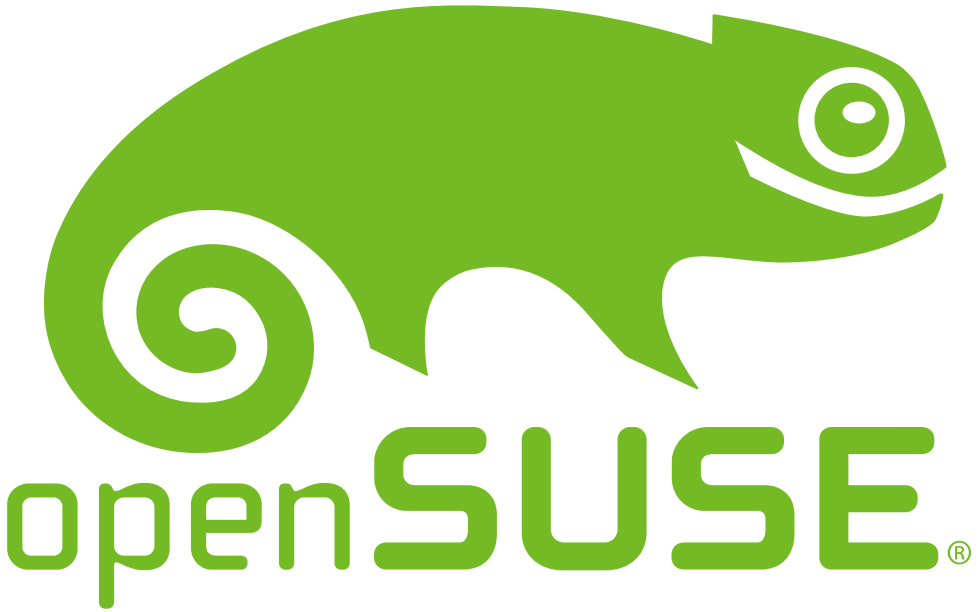
Green Computing from the CLI

Measuring Energy Consumption on Linux 🔋🐧

Andrea Manzini

Software Quality Engineer @ SUSE

andrea.manzini@suse.com



About Me

- Veteran Unix Admin / BOFH background
- QA Engineer @SUSE
- Open Source Contributor
- OpenSUSE Package Maintainer
- 🌍 Passionate about efficiency and sustainability

andrea.manzini@suse.com

Why Energy Efficiency?

It's not just "being green"

- **Mobile & IoT:** Battery life is a competitive advantage.
- **Data Centers:** Small savings $\times$ thousands of servers = Millions of € and tons of CO_2.
- **Performance:** High energy = Heat = *Thermal Throttling*.
- **The "Cloud" Gap:** "If I can't touch the hardware, how do I measure it?"

Basics math: Power vs. Energy

Power [Watt]:

The *rate* at which energy is consumed.
Analogy: Car speed (90 km/h).

Energy [Joule]:

The *total* amount used over time.

Energy = Power × Time

Analogy: Distance traveled (200 km).

Our Goal: Minimize the total Joules for a specific task.

andrea.manzini@suse.com

The Hardware Magic: Intel RAPL

Running Average Power Limit

Modern CPUs don't have physical meters inside, but they have accurate **Power Models**.

- **Domains:**
  - **PKG:** Entire CPU socket.
  - **CORE:** Computation cores.
  - **DRAM:** Memory controller & RAM.
- **Accuracy:** Precise estimation based on hardware counters, voltage, and temperature.

Measuring from Code: powercap

Linux exposes RAPL via `/sys/class/powercap/intel-rapl/`. Let's build a Python decorator:

```
import time

def read_uj(): return int(open("/sys/class/powercap/intel-rapl:0/energy_uj").read())

def measure_energy(f):
    def wrapper(*args, **kwargs):
        e_start, t_start = read_uj(), time.time()
        res = f(*args, **kwargs)
        joules = (read_uj() - e_start) / 1e6
        print(f"[{f.__name__}] {joules:.2f}J in {time.time()-t_start:.2f}s")
        return res
    return wrapper
```

Using the Energy Decorator

```
@measure_energy
def process_data(records):
    # Heavy computation here
    return sorted(records, key=lambda x: x['value'])

# Output:
# [process_data] 12.45 Joules in 0.85s
```

Perfect for local benchmarking of specific functions!

The CLI Toolbox

Tool	Best For...	Command
powertop	System-wide diagnosis & "vampire" processes.	`sudo powertop`
powerstat	Real-time monitoring of system drain.	`sudo powerstat`
perf	**Precision surgical measurement** of a command.	`sudo perf stat -e power/energy-pkg/`

Experiment: Sorting a 1GB text file

Let's create a 1GB text file:

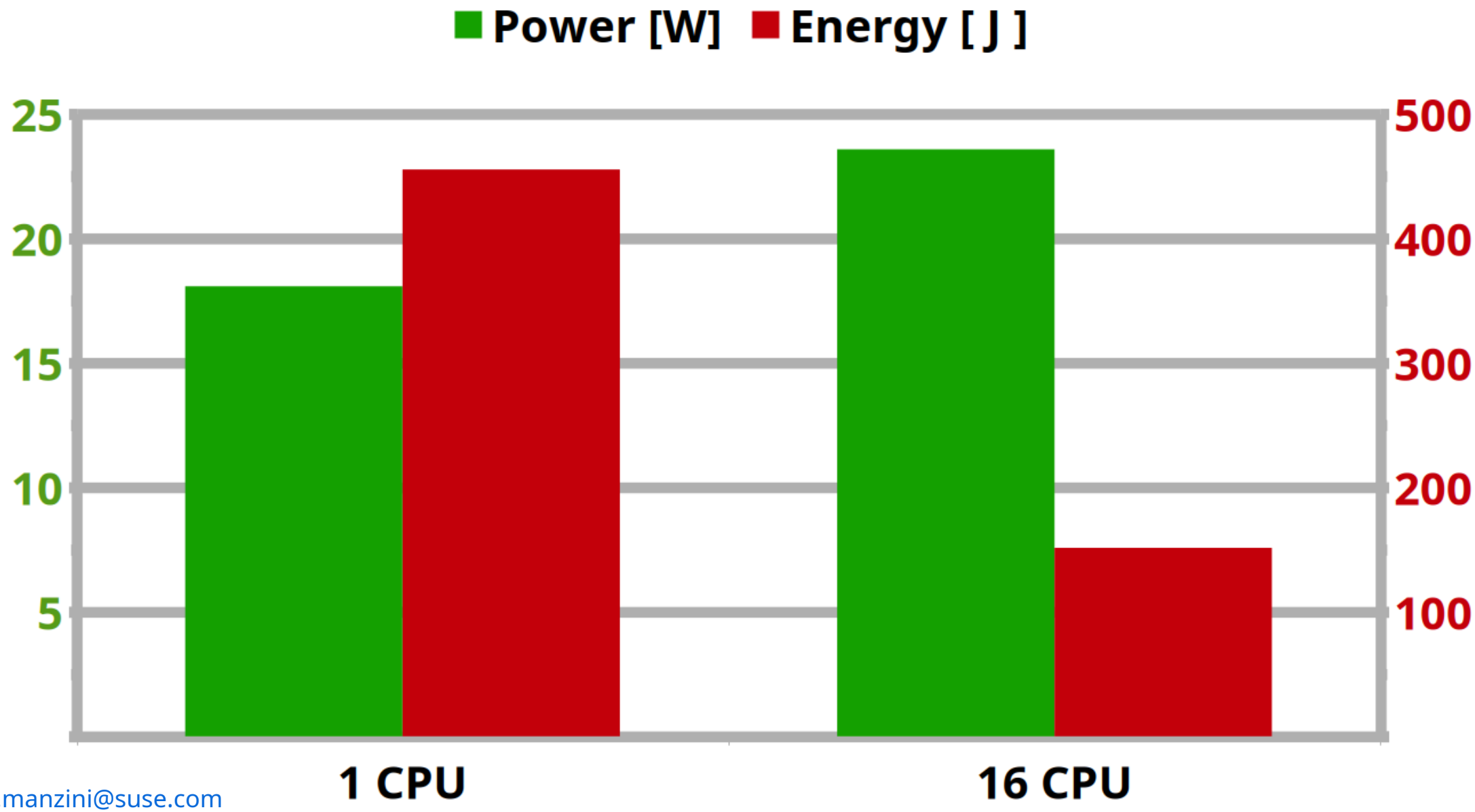
```
$ tr -dc 'A-Za-z0-9 ' < /dev/urandom | fold -w 80 | head -c 1G > big.txt
```

Now we want it sorted!

```
# Single-threaded
$ sudo perf stat -e power/energy-pkg/ -- sort --parallel=1 big.txt > /dev/null
# 25s @ ~18W = 455 Joules

# 16 Threads
$ sudo perf stat -e power/energy-pkg/ -- sort --parallel=16 big.txt > /dev/null
# 6.4s @ ~24W = 151 Joules
```

Result: ~66% Energy Saving [just by adding an option]


Power [W]
Energy [J]
25
20
15
10
5
500
400
300
200
100
1 CPU
16 CPU

Experiment: Python vs Rust

Calculate the 1,000,000th prime number, using the same algorithm.

Python

```
def is_prime(n):
    if n <= 1: return False
    if n <= 3: return True
    if n % 2 == 0 or n % 3 == 0: return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True
```

Rust

```rust
fn is_prime(n: u64) -> bool {
    if n <= 1 { return false; }
    if n <= 3 { return true; }
    if n % 2 == 0 || n % 3 == 0 { return false; }
    let mut i = 5;
    while i * i <= n {
        if n % i == 0 || n % (i + 2) == 0 {
            return false;
        }
        i += 6;
    }
    true
}
```

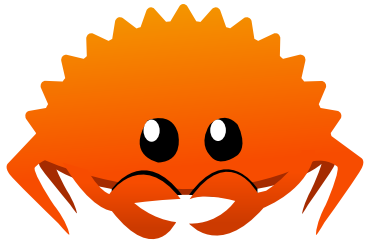
Results

Language	Nr	Time	Energy (J)
Python	15485863	50.2 s	934 J
Rust	15485863	2.1 s	39 J

Rust is 23x more energy-efficient.

Why?

- No Garbage Collector overhead
- Ahead-of-Time (AOT) compilation
- Better data locality and zero-cost abstractions



Catching Regressions in CI/CD

Standard cloud runners (VMs) lack power meters. Instead, track **proxy metrics** like CPU Instructions or Cycles.

```
# .github/workflows/energy-test.yml
steps:
  - run: cargo build --release
  - name: Measure Proxy Metrics
    run: |
      sudo perf stat -e instructions -x, -o perf.csv \
        ./target/release/my_app --benchmark

      INSTR=$(awk -F, '/instructions/ {print $1}' perf.csv)

      # Fail build if instruction count spikes
      if (( $(echo "$INSTR > 500000000" | bc -l) )); then
        echo "Efficiency regression! $INSTR instructions used."
        exit 1
      fi
```

Beyond Local: The cloud-ops view

On the cloud, hypervisors hide RAPL. How do we measure containers?

- **Kepler (eBPF):** Watches CPU instructions and cache misses. Uses ML to map them to power consumption per Kubernetes Pod.
- **Scaphandre:** Rust agent bridging low-level metrics to Prometheus. Includes "Embodied Carbon" estimations.
- **OpenTelemetry:** Energy is being standardized as a metric. APM tools will show Energy by default.

Shift your focus to "Joules per Request".

Jevons' paradox (The Rebound Effect)

Why efficiency isn't enough.

As technological progress increases efficiency, the rate of consumption rises due to increasing demand.

- **Example:** We moved to highly efficient SSDs. Result? We started storing 100x more data because it became cheap and fast.
- **The AI Era:** More efficient GPUs don't mean less energy used; they mean we train *larger* models.
- **The Takeaway:** Technical efficiency must be paired with conscious design.

What Can YOU Do?

1. **Optimize Hot Paths:** Benchmark and optimize the loops that run millions of times.
2. **Choose the Right Tool:** Don't write CPU-bound microservices in interpreted languages if throughput/energy matters.
3. **CI/CD Gates:** Track energy metrics just like code coverage or execution time.
4. **Demand Transparency:** Ask cloud providers about energy metrics and carbon intensity.

Summary & Takeaways

1. **Efficiency = Quality:** Green code is often just *good* code.
2. **Measure in Code:** Build simple wrappers around `powercap`.
3. **Automate It:** Put `perf stat` in your CI pipeline.
4. **Race to Sleep:** Finish work quickly so hardware can idle.

Thanks for your attention!

Andrea Manzini

https://ilmanzo.github.io

Mastodon/GitHub: `@ilmanzo`

Questions?